МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

ЗВІТ

з лабораторної роботи №5

з дисципліни «Об'єктно-орієнтоване програмування»

Тема: «Розробка багатовіконного інтерфейсу користувача для графічного редактора об'єктів»

Виконав:
студент 2-го курсу групи ІМ-О51
Конкус Андрій Андрійович
Номер за списком у журналі: 4

Перевірив:
викладач кафедри ОТ
Рекечинський Дмитро Олександрович

Київ — 2026

1. МЕТА РОБОТИ ТА РОЗРАХУНОК ВАРІАНТУ

Мета: Отримати вміння та навички програмувати багатовіконний інтерфейс програми в об'єктно-орієнтованому стилі.

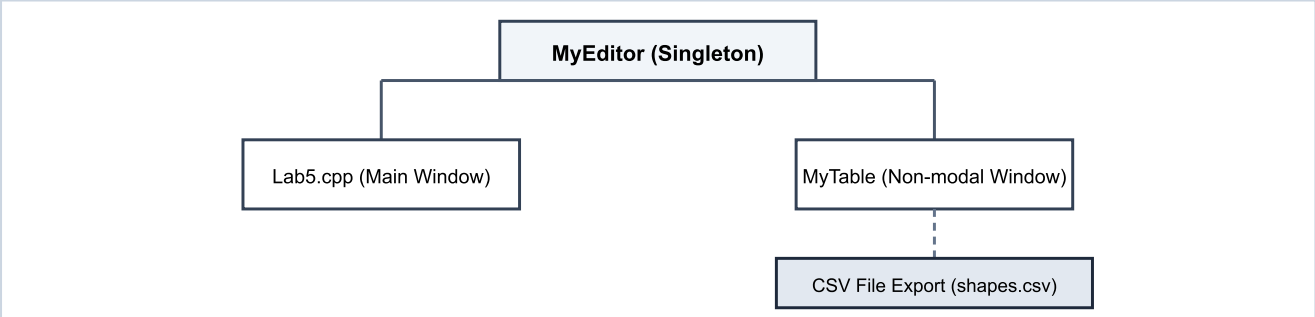
Розрахунок варіанту для K = 4 (Парний номер):

Параметр завдання	**Правило / Формула (K=4)**	**Результат розрахунку та вимоги**
Паттерн Singleton	Парний номер (2, 4, ...)	Реалізація об'єкта класу `MyEditor` на основі класичної реалізації Singleton (приватний конструктор, статика `GetInstance()`).
Вікно таблиці об'єктів	Пункт 3 завдання	Немодальне вікно діалогу `MyTable` для відображення списку об'єктів та їх координат (x1, y1, x2, y2).
Керування вікном таблиці	Пункт 3 завдання	Відкриття/закриття через пункт меню («Table Window»). Автоматичне закриття при виході з програми.
Запис у файл	Пункт 5 завдання	Запис списку введених об'єктів у файл формату CSV («Save to CSV»).

2. ОСОБЛИВОСТІ ІНТЕРФЕЙСУ ТА АРХІТЕКТУРИ

- **Паттерн Singleton:** Гарантує існування єдиного екземпляра класу редактора `MyEditor` у всій системі. Доступ до редактора здійснюється через статичний метод.
- **Немодальне вікно MyTable:** Створюється за допомогою `CreateDialogParam`, що дозволяє користувачеві одночасно малювати фігури у головному вікні та спостерігати за їх координатами у таблиці.
- **Збереження даних (CSV):** Реалізовано діалогове вікно вибору файлу `GetSaveFileName` та збереження параметрів графічних об'єктів у текстовий файл з розділювачами.

3. СХЕМА УСПАДКУВАННЯ ТА СТРУКТУРА МОДУЛІВ



5. ІЛЮСТРАЦІЇ РОБОТИ ПРОГРАМИ

Нижче наведено скріншоти виконання програми, що демонструють функціонал багатовіконного інтерфейсу, таблиці об'єктів та збереження у CSV файл:

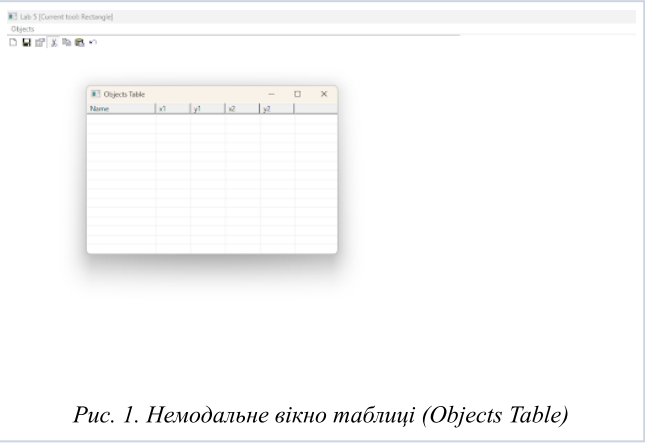


Рис. 1. Немодальне вікно таблиці (Objects Table)

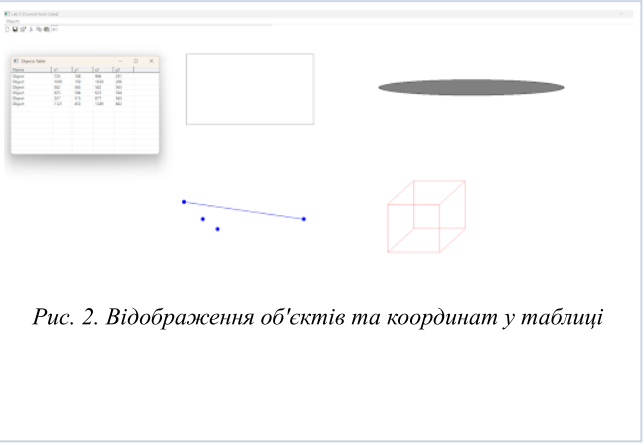


Рис. 2. Відображення об'єктів та координат у таблиці

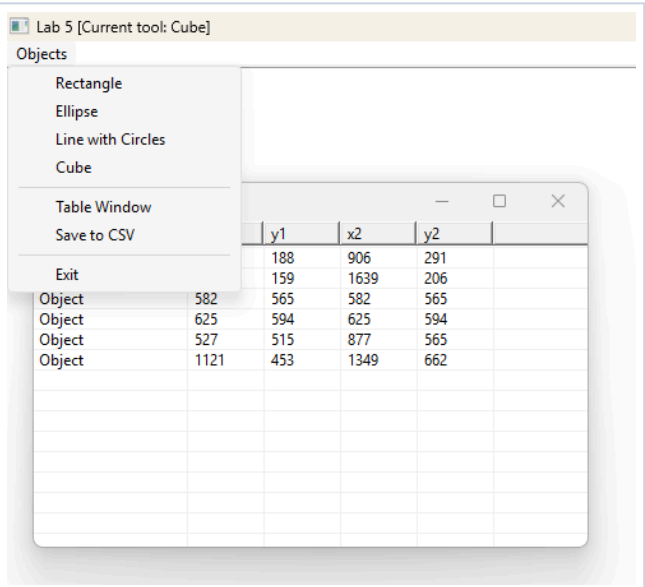


Рис. 3. Пункти меню для відкриття таблиці та збереження

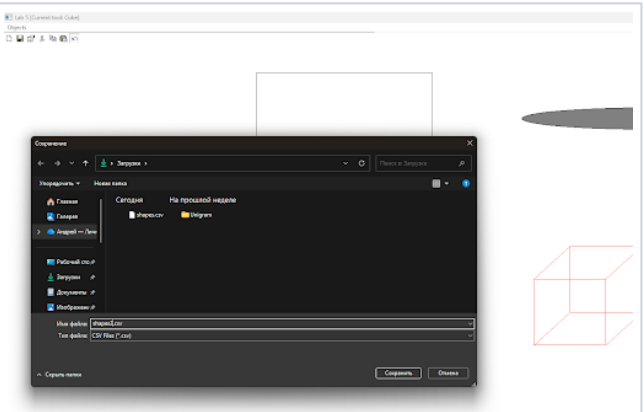


Рис. 4. Стандартний діалог збереження CSV файлу

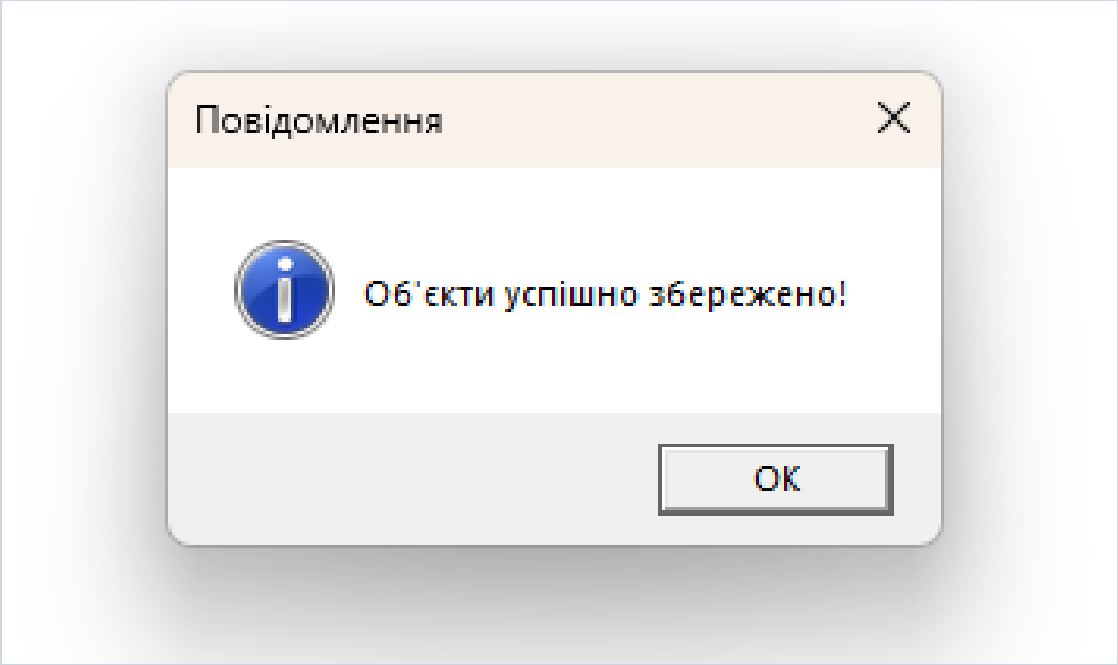


Рис. 5. Повідомлення про успішне збереження об'єктів

6. ВИСНОВКИ

Під час виконання лабораторної роботи №5 було успішно розроблено багатовіконний інтерфейс користувача для графічного редактора об'єктів у середовищі Win32 API.

В результаті виконання роботи:

- Реалізовано класичний варіант паттерну **Singleton** для гарантії єдиного екземпляра головного редактора `MyEditor`.
- Створено окремий модуль та клас `MyTable`, який керує немодальним вікном таблиці для оперативної індикації списку та координат введених об'єктів.
- Забезпечено синхронну взаємодію між головним вікном малювання та вікном таблиці, а також реалізовано автоматичне закриття немодального вікна при завершенні програми.
- Реалізовано функціонал експорту списку створених об'єктів у файл формату CSV за допомогою системного діалогу вибору файлу.

4. ТЕКСТ ПРОГРАМИ (ЛІСТИНГ КОДУ)

Нижче наведено повний реалізований вихідний код модуля табличного відображення об'єктів та головного модуля програми Visual C++ WinAPI.

4.1. Заголовочний файл my_table.h

```
#ifndef MY_TABLE_H
#define MY_TABLE_H

#include <windows.h>
#include <commctrl.h>
#include "shape.h"

class MyTable {
private:
    HWND hWndTable;
    HWND hWndList;
public:
    MyTable();
    void Show(HWND hWndParent);
    void Hide();
    void Update(Shape** pcshape, int shapeCount);
    HWND GetHWND() const { return hWndTable; }
};

#endif
```

4.2. Файл реалізації my_table.cpp

```
#include "my_table.h"
#include <string>

static const wchar_t TABLE_CLASS_NAME[] = L"MyTableWindowClass";

LRESULT CALLBACK TableWndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {
    case WM_CLOSE:
        ShowWindow(hWnd, SW_HIDE);
        return 0;
    case WM_SIZE: {
        HWND hWndList = GetDlgItem(hWnd, 1001);
        if (hWndList) {
            MoveWindow(hWndList, 0, 0, LOWORD(lParam), HIWORD(lParam), TRUE);
        }
    }
        break;
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

MyTable::MyTable() {
    hWndTable = NULL;
    hWndList = NULL;
}

void MyTable::Show(HWND hWndParent) {
    if (!hWndTable) {
        WNDCLASSW wc = {};
        wc.lpfnWndProc = TableWndProc;
        wc.hInstance = GetModuleHandle(NULL);
        wc.lpszClassName = TABLE_CLASS_NAME;
```

```
        wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wc.hCursor = LoadCursor(NULL, IDC_ARROW);
        RegisterClassW(&wc);

        hWndTable = CreateWindowExW(
            0, TABLE_CLASS_NAME, L"Objects Table",
            WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 450, 300,
            hWndParent, NULL, GetModuleHandle(NULL), NULL
        );

        RECT rc;
        GetClientRect(hWndTable, &rc);

        hWndList = CreateWindowExW(
            0, WC_LISTVIEWW, L"",
            WS_CHILD | WS_VISIBLE | LVS_REPORT | LVS_SINGLESEL,
            0, 0, rc.right, rc.bottom,
            hWndTable, (HMENU)1001, GetModuleHandle(NULL), NULL
        );

        ListView_SetExtendedListViewStyle(hWndList, LVS_EX_FULLROWSELECT | LVS_EX_GRIDLINES);

        LVCOLUMNW lvc = {};
        lvc.mask = LVCF_TEXT | LVCF_WIDTH | LVCF_SUBITEM;

        lvc.iSubItem = 0; lvc.pszText = (LPWSTR)L"Name"; lvc.cx = 120;
        SendMessageW(hWndList, LVM_INSERTCOLUMN, 0, (LPARAM)&lvc);

        lvc.iSubItem = 1; lvc.pszText = (LPWSTR)L"X1"; lvc.cx = 60;
        SendMessageW(hWndList, LVM_INSERTCOLUMN, 1, (LPARAM)&lvc);

        lvc.iSubItem = 2; lvc.pszText = (LPWSTR)L"Y1"; lvc.cx = 60;
        SendMessageW(hWndList, LVM_INSERTCOLUMN, 2, (LPARAM)&lvc);

        lvc.iSubItem = 3; lvc.pszText = (LPWSTR)L"X2"; lvc.cx = 60;
        SendMessageW(hWndList, LVM_INSERTCOLUMN, 3, (LPARAM)&lvc);

        lvc.iSubItem = 4; lvc.pszText = (LPWSTR)L"Y2"; lvc.cx = 60;
        SendMessageW(hWndList, LVM_INSERTCOLUMN, 4, (LPARAM)&lvc);
    }
    ShowWindow(hWndTable, SW_SHOW);
}

void MyTable::Hide() {
    if (hWndTable) ShowWindow(hWndTable, SW_HIDE);
}

void MyTable::Update(Shape** pcshape, int shapeCount) {
    if (!hWndList) return;

    SendMessageW(hWndList, LVM_DELETEALLITEMS, 0, 0);

    for (int i = 0; i < shapeCount; i++) {
        if (!pcshape[i]) continue;

        wchar_t buf1[16], buf2[16], buf3[16], buf4[16];
        swprintf_s(buf1, L"%d", pcshape[i]->GetX1());
        swprintf_s(buf2, L"%d", pcshape[i]->GetY1());
        swprintf_s(buf3, L"%d", pcshape[i]->GetX2());
        swprintf_s(buf4, L"%d", pcshape[i]->GetY2());

        LVITEMW lvi = {};
        lvi.mask = LVIF_TEXT;
        lvi.iItem = i;
        lvi.iSubItem = 0;
        lvi.pszText = (LPWSTR)L"Object";
        SendMessageW(hWndList, LVM_INSERTITEM, 0, (LPARAM)&lvi);
```

```
        LVITEMW subitem = {};
        subitem.iItem = i;

        subitem.iSubItem = 1; subitem.pszText = buf1;
        SendMessageW(hWndList, LVM_SETITEMTEXT, i, (LPARAM)&subitem);

        subitem.iSubItem = 2; subitem.pszText = buf2;
        SendMessageW(hWndList, LVM_SETITEMTEXT, i, (LPARAM)&subitem);

        subitem.iSubItem = 3; subitem.pszText = buf3;
        SendMessageW(hWndList, LVM_SETITEMTEXT, i, (LPARAM)&subitem);

        subitem.iSubItem = 4; subitem.pszText = buf4;
        SendMessageW(hWndList, LVM_SETITEMTEXT, i, (LPARAM)&subitem);
    }
}
```

4.3. Головний файл Lab5.cpp

```
#include <windows.h>
#define _WIN32_IE 0x0500
#include <commctrl.h>
#include <fstream>
#include "resource.h"
#include "my_table.h"

#include "rect_editor.h"
#include "ellipse_editor.h"
#include "line_oo_editor.h"
#include "cube_editor.h"

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "comctl32.lib")
#pragma comment(lib, "MyEditorDll.lib")

class MyEditor {
public:
    enum ToolType { TOOL_RECT, TOOL_ELLIPSE, TOOL_LINEOO, TOOL_CUBE };

private:
    static MyEditor* pInstance;
    Shape* pcshape[105];
    int shapeCount;

    RectEditor rectEditor;
    EllipseEditor ellipseEditor;
    LineOOEditor lineOOEditor;
    CubeEditor cubeEditor;

    ToolType currentTool;
    MyTable tableWindow;

    MyEditor() {
        shapeCount = 0;
        currentTool = TOOL_RECT;
        for (int i = 0; i < 105; i++) pcshape[i] = nullptr;
    }

public:
    static MyEditor* getInstance() {
        if (!pInstance) {
            pInstance = new MyEditor();
        }
        return pInstance;
```

```
    }

    static void destroyInstance() {
        if (pInstance) {
            delete pInstance;
            pInstance = nullptr;
        }
    }

    ~MyEditor() {
        for (int i = 0; i < shapeCount; i++) {
            if (pcshape[i]) delete pcshape[i];
        }
    }

    void SetTool(ToolType tool) { currentTool = tool; }
    ToolType GetTool() const { return currentTool; }

    void OnLButtonDown(HWND hWnd, LPARAM lParam) {
        if (currentTool == TOOL_RECT) rectEditor.OnLButtonDown(hWnd, lParam);
        else if (currentTool == TOOL_ELLIPSE) ellipseEditor.OnLButtonDown(hWnd, lParam);
        else if (currentTool == TOOL_LINEOO) lineOOEditor.OnLButtonDown(hWnd, lParam);
        else if (currentTool == TOOL_CUBE) cubeEditor.OnLButtonDown(hWnd, lParam);
    }

    void OnMouseMove(HWND hWnd, LPARAM lParam) {
        if (currentTool == TOOL_RECT) rectEditor.OnMouseMove(hWnd, lParam);
        else if (currentTool == TOOL_ELLIPSE) ellipseEditor.OnMouseMove(hWnd, lParam);
        else if (currentTool == TOOL_LINEOO) lineOOEditor.OnMouseMove(hWnd, lParam);
        else if (currentTool == TOOL_CUBE) cubeEditor.OnMouseMove(hWnd, lParam);
    }

    void OnLButtonUp(HWND hWnd, LPARAM lParam) {
        if (currentTool == TOOL_RECT) rectEditor.OnLButtonUp(hWnd, lParam, pcshape, shapeCount);
        else if (currentTool == TOOL_ELLIPSE) ellipseEditor.OnLButtonUp(hWnd, lParam, pcshape,
shapeCount);
        else if (currentTool == TOOL_LINEOO) lineOOEditor.OnLButtonUp(hWnd, lParam, pcshape, shapeCount);
        else if (currentTool == TOOL_CUBE) cubeEditor.OnLButtonUp(hWnd, lParam, pcshape, shapeCount);

        tableWindow.Update(pcshape, shapeCount);
    }

    void OnPaint(HDC hdc) {
        for (int i = 0; i < shapeCount; i++) {
            if (pcshape[i]) pcshape[i]->Draw(hdc);
        }
    }

    void ToggleTable(HWND hWndParent) {
        tableWindow.Show(hWndParent);
        tableWindow.Update(pcshape, shapeCount);
    }

    void SaveToCSV(HWND hWndParent) {
        wchar_t szFile[260] = L"shapes.csv";
        OPENFILENAMEW ofn = {};
        ofn.lStructSize = sizeof(ofn);
        ofn.hwndOwner = hWndParent;
        ofn.lpstrFilter = L"CSV Files (*.csv)\0*.csv\0All Files (*.*)\0*.*\0";
        ofn.lpstrFile = szFile;
        ofn.nMaxFile = sizeof(szFile) / sizeof(wchar_t);
        ofn.nFilterIndex = 1;
        ofn.Flags = OFN_PATHMUSTEXIST | OFN_OVERWRITEPROMPT;
        ofn.lpstrDefExt = L"csv";

        if (GetSaveFileNameW(&ofn)) {
            std::ofstream file(ofn.lpstrFile);
```

```
            if (file.is_open()) {
                file << "Name,x1,y1,x2,y2\n";
                for (int i = 0; i < shapeCount; i++) {
                    if (pcshape[i]) {
                        file << "Object,"
                             << pcshape[i]->GetX1() << ","
                             << pcshape[i]->GetY1() << ","
                             << pcshape[i]->GetX2() << ","
                             << pcshape[i]->GetY2() << "\n";
                    }
                }
                file.close();
                MessageBoxW(hWndParent, L"Об'єкти успішно збережено!", L"Повідомлення", MB_OK |
MB_ICONINFORMATION);
            }
        }
    }

    void LoadFromCSV(HWND hWndParent) {
        wchar_t szFile[260] = L"";
        OPENFILENAMEW ofn = {};
        ofn.lStructSize = sizeof(ofn);
        ofn.hwndOwner = hWndParent;
        ofn.lpstrFilter = L"CSV Files (*.csv)\0*.csv\0All Files (*.*)\0*.*\0";
        ofn.lpstrFile = szFile;
        ofn.nMaxFile = sizeof(szFile) / sizeof(wchar_t);
        ofn.nFilterIndex = 1;
        ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

        if (GetOpenFileNameW(&ofn)) {
            std::ifstream file(ofn.lpstrFile);
            if (file.is_open()) {
                file.close();
                tableWindow.Update(pcshape, shapeCount);
                InvalidateRect(hWndParent, NULL, TRUE);
                MessageBoxW(hWndParent, L"Файл прочитано!", L"Загрузка", MB_OK | MB_ICONINFORMATION);
            }
        }
    }
};

MyEditor* MyEditor::pInstance = nullptr;
static HWND hWndToolBar = NULL;

void UpdateWindowTitle(HWND hWnd) {
    MyEditor* editor = MyEditor::getInstance();
    switch (editor->GetTool()) {
    case MyEditor::TOOL_RECT: SetWindowTextW(hWnd, L"Lab 5 [Current tool: Rectangle]"); break;
    case MyEditor::TOOL_ELLIPSE: SetWindowTextW(hWnd, L"Lab 5 [Current tool: Ellipse]"); break;
    case MyEditor::TOOL_LINEOO: SetWindowTextW(hWnd, L"Lab 5 [Current tool: Line with Circles]"); break;
    case MyEditor::TOOL_CUBE: SetWindowTextW(hWnd, L"Lab 5 [Current tool: Cube]"); break;
    }
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    MyEditor* editor = MyEditor::getInstance();

    switch (message) {
    case WM_CREATE: {
        INITCOMMONCONTROLSEX icex;
        icex.dwSize = sizeof(INITCOMMONCONTROLSEX);
        icex.dwICC = ICC_BAR_CLASSES | ICC_LISTVIEW_CLASSES;
        InitCommonControlsEx(&icex);

        hWndToolBar = CreateWindowExW(0, TOOLBARCLASSNAME, NULL,
            WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT | TBSTYLE_TOOLTIPS,
            0, 0, 0, 0, hWnd, (HMENU)100, GetModuleHandle(NULL), NULL);
```

```
    SendMessage(hWndToolBar, TB_BUTTONSTRUCTSIZE, (WPARAM)sizeof(TBBUTTON), 0);

    TBADDBITMAP tbab;
    tbab.hInst = HINST_COMMCTRL;
    tbab.nID = 0;
    SendMessage(hWndToolBar, TB_ADDBITMAP, 0, (LPARAM)&tbab);

    TBBUTTON tbb[7] = {};
    tbb[0].iBitmap = STD_FILENEW;
    tbb[0].idCommand = IDM_SAVE;
    tbb[0].fsState = TBSTATE_ENABLED;
    tbb[0].fsStyle = BTNS_BUTTON;

    tbb[1].iBitmap = STD_FILESAVE;
    tbb[1].idCommand = IDM_SAVE;
    tbb[1].fsState = TBSTATE_ENABLED;
    tbb[1].fsStyle = BTNS_BUTTON;

    tbb[2].iBitmap = STD_PROPERTIES;
    tbb[2].idCommand = IDM_TABLE;
    tbb[2].fsState = TBSTATE_ENABLED;
    tbb[2].fsStyle = BTNS_BUTTON;

    tbb[3].iBitmap = 0;
    tbb[3].idCommand = IDM_RECTANGLE;
    tbb[3].fsState = TBSTATE_ENABLED | TBSTATE_CHECKED;
    tbb[3].fsStyle = BTNS_CHECK;

    tbb[4].iBitmap = 1;
    tbb[4].idCommand = IDM_ELLIPSE;
    tbb[4].fsState = TBSTATE_ENABLED;
    tbb[4].fsStyle = BTNS_CHECK;

    tbb[5].iBitmap = 2;
    tbb[5].idCommand = IDM_LINEOO;
    tbb[5].fsState = TBSTATE_ENABLED;
    tbb[5].fsStyle = BTNS_CHECK;

    tbb[6].iBitmap = 3;
    tbb[6].idCommand = IDM_CUBE;
    tbb[6].fsState = TBSTATE_ENABLED;
    tbb[6].fsStyle = BTNS_CHECK;

    SendMessage(hWndToolBar, TB_ADDBUTTONS, 7, (LPARAM)&tbb);
    SendMessage(hWndToolBar, TB_AUTOSIZE, 0, 0);
}
break;

case WM_COMMAND: {
    int wmId = LOWORD(wParam);
    switch (wmId) {
    case IDM_RECTANGLE:
        editor->SetTool(MyEditor::TOOL_RECT);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_RECTANGLE, TRUE);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_ELLIPSE, FALSE);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_LINEOO, FALSE);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_CUBE, FALSE);
        UpdateWindowTitle(hWnd);
        break;

    case IDM_ELLIPSE:
        editor->SetTool(MyEditor::TOOL_ELLIPSE);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_RECTANGLE, FALSE);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_ELLIPSE, TRUE);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_LINEOO, FALSE);
        SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_CUBE, FALSE);
```

```
            UpdateWindowTitle(hWnd);
            break;

        case IDM_LINEOO:
            editor->SetTool(MyEditor::TOOL_LINEOO);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_RECTANGLE, FALSE);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_ELLIPSE, FALSE);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_LINEOO, TRUE);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_CUBE, FALSE);
            UpdateWindowTitle(hWnd);
            break;

        case IDM_CUBE:
            editor->SetTool(MyEditor::TOOL_CUBE);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_RECTANGLE, FALSE);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_ELLIPSE, FALSE);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_LINEOO, FALSE);
            SendMessage(hWndToolBar, TB_CHECKBUTTON, IDM_CUBE, TRUE);
            UpdateWindowTitle(hWnd);
            break;

        case IDM_TABLE:
            editor->ToggleTable(hWnd);
            break;

        case IDM_SAVE:
            editor->SaveToCSV(hWnd);
            break;

        case IDM_EXIT:
            DestroyWindow(hWnd);
            break;

        default:
            return DefWindowProc(hWnd, message, wParam, lParam);
        }
    }
    break;

    case WM_LBUTTONDOWN: editor->OnLButtonDown(hWnd, lParam); break;
    case WM_MOUSEMOVE: editor->OnMouseMove(hWnd, lParam); break;
    case WM_LBUTTONUP: editor->OnLButtonUp(hWnd, lParam); break;

    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        editor->OnPaint(hdc);
        EndPaint(hWnd, &ps);
    }
    break;

    case WM_DESTROY:
        MyEditor::destroyInstance();
        PostQuitMessage(0);
        break;

    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

int WINAPI WinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ LPSTR lpCmdLine, _In_
int nCmdShow) {
    const wchar_t CLASS_NAME[] = L"Lab5WindowClass";

    WNDCLASSW wc = {};
```

```
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.hCursor = LoadCursor(NULL, IDC_CROSS);

    RegisterClassW(&wc);

    HMENU hMenu = LoadMenuW(hInstance, MAKEINTRESOURCEW(IDR_MYMENU));

    HWND hWnd = CreateWindowExW(
        0, CLASS_NAME, L"Lab 5 [Current tool: Rectangle]",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 800, 600,
        NULL, hMenu, hInstance, NULL
    );

    if (hWnd == NULL) return 0;

    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);

    MSG msg = {};
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return (int)msg.wParam;
}
```